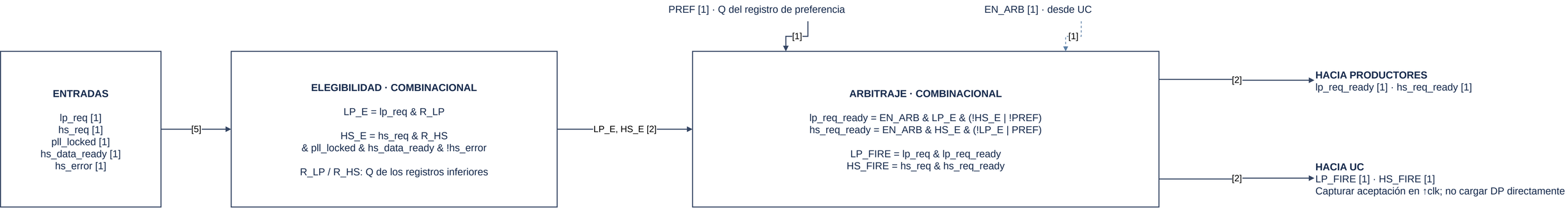
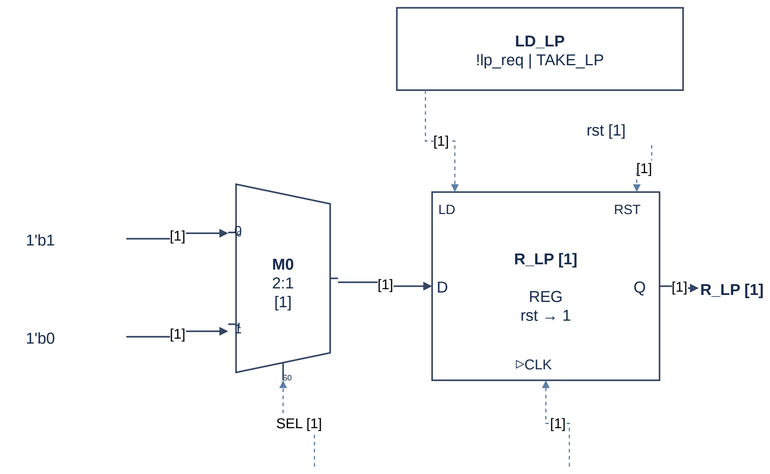
DPHY_MODE_CTRL · DATAPATH · ARBITRAJE Y REARME

RTL · DATAPATH 1 / 2
Registros ↓clk · UC ↑clk · 40 MHz · buses [n]. Líneas azules discontinuas: control / reloj / reset. Nombres repetidos: misma red.



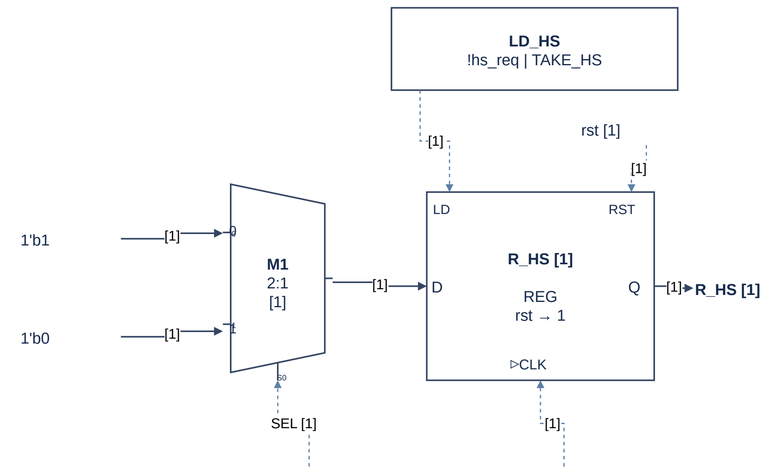
CONTRATO DEL ARBITRAJE
EN_ARB = !rst & ARB_LIBRE & stop_ready. ARB_LIBRE lo entrega la unidad de control.
PREF = 0 prefiere LP en empate; PREF = 1 prefiere HS. Solo una reserva puede ocurrir por flanco.
LP_E / HS_E son condiciones; LP_FIRE / HS_FIRE son eventos de aceptación, distintos de los grants de propiedad.

REARME DE SOLICITUD LP



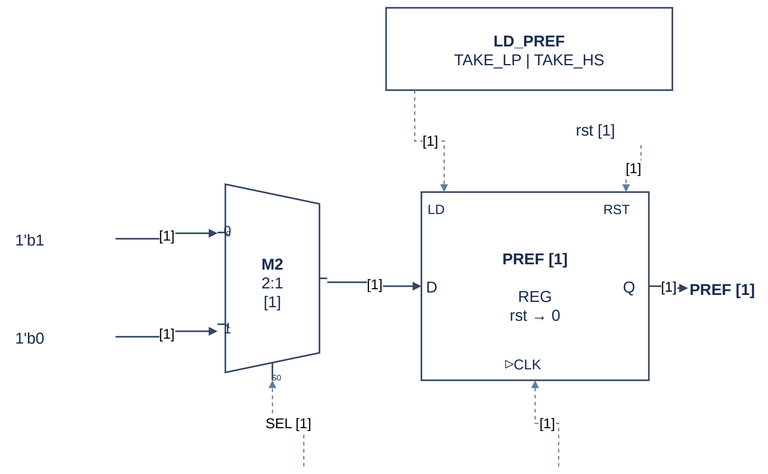
En ↓clk, si lp_req = 0: R_LP ← 1.
Si sigue alto y TAKE_LP = 1: R_LP ← 0.
TAKE procede de la acción de ↑clk; sin carga conservar Q.

REARME DE SOLICITUD HS



En ↓clk, si hs_req = 0: R_HS ← 1.
Si sigue alto y TAKE_HS = 1: R_HS ← 0.
TAKE procede de la acción de ↑clk; sin carga conservar Q.

PREFERENCIA DE EMPATE



En ↓clk: aceptar LP deja PREF = 1;
aceptar HS deja PREF = 0. TAKE viene de ACTION.
Sin acción de aceptación: conservar Q.

ENTRADAS A UC

LP_FIRE, HS_FIRE, HS_FAULT [1 cada una]
lp_done, hs_done, clk_prepared, data_prepared
hs_data_ready, data_stopped, clk_stopped
stop_ready, clear_error, clk, rst [1 cada una]

condiciones / eventos

UNIDAD DE CONTROL · ↑clk

STATE [4]: fase. ACTION [4]: decisión retenida hasta ↓clk.
Transiciones en always @(posedge clk).
Salidas en always @(*).

Ocho bits de UC, separados de los siete del datapath.
Contrato temporal completo: página 9.

controles

SALIDAS COMBINACIONALES DE UC

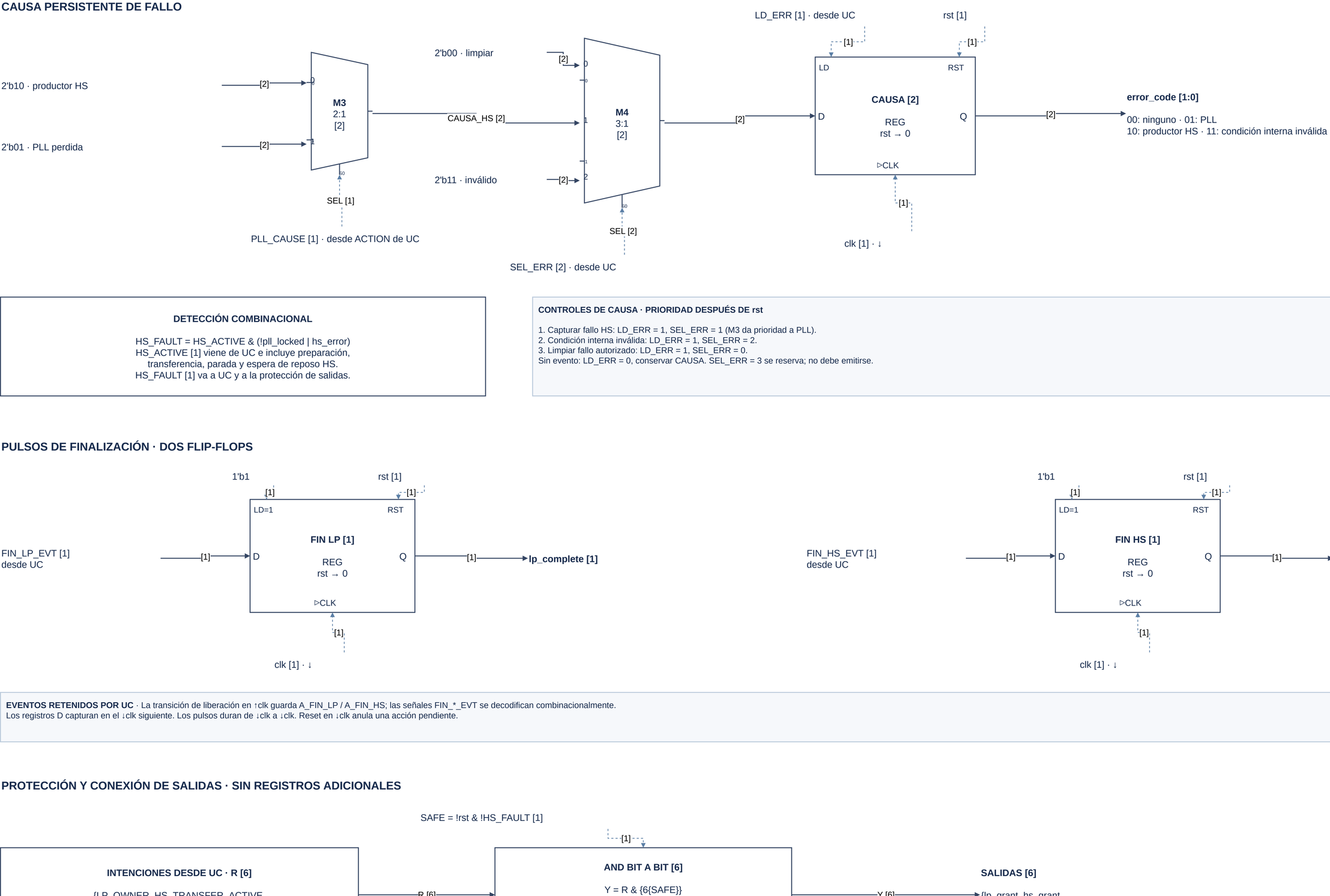
ARB_LIBRE → EN_ARB [1]; TAKE_LP / TAKE_HS [1]
LD_ERR [1], SEL_ERR [2], PLL_CAUSE [1]
FIN_LP_EVT / FIN_HS_EVT [1]
HS_ACTIVE, FAULT_ACTIVE, BUSY_CTRL [1]; intenciones [6]

REGLAS EN ↓clk: rst tiene prioridad sobre LD; LD = 0 conserva Q.
M0/M1 rearman si req ya está bajo; si está alto, TAKE consume. M2 usa la aceptación retenida. No cargar desde FIRE vivo tras cambiar de fase.

DPHY_MODE_CTRL · DATAPATH · FALLOS, FIN Y SALIDAS

RTL · DATAPATH 2 / 2
Registros ↓clk · UC ↑clk · 40 MHz. Causa [2], pulsos [1 + 1]. La acción decidida en ↑clk conserva evento y causa hasta ↓clk.

CAUSA PERSISTENTE DE FALLO



DETECCIÓN COMBINACIONAL

HS_FAULT = HS_ACTIVE & (!pll_locked | hs_error)
HS_ACTIVE [1] viene de UC e incluye preparación, transferencia, parada y espera de reposo HS.
HS_FAULT [1] va a UC y a la protección de salidas.

CONTROLES DE CAUSA · PRIORIDAD DESPUÉS DE rst

1. Capturar fallo HS: LD_ERR = 1, SEL_ERR = 1 (M3 da prioridad a PLL).
2. Condición interna inválida: LD_ERR = 1, SEL_ERR = 2.
3. Limpiar fallo autorizado: LD_ERR = 1, SEL_ERR = 0.
Sin evento: LD_ERR = 0, conservar CAUSA. SEL_ERR = 3 se reserva; no debe emitirse.

PULSOS DE FINALIZACIÓN · DOS FLIP-FLOPS

EVENTOS RETENIDOS POR UC · La transición de liberación en ↑clk guarda A_FIN_LP / A_FIN_HS; las señales FIN_*_EVT se decodifican combinacionalmente.
Los registros D capturan en el ↓clk siguiente. Los pulsos duran de ↓clk a ↓clk. Reset en ↓clk anula una acción pendiente.

PROTECCIÓN Y CONEXIÓN DE SALIDAS · SIN REGISTROS ADICIONALES

MEMORIA: datapath [7] en ↓clk + STATE [4] y ACTION [4] de UC en ↑clk = 15 bits lógicos.
La causa PLL/HS se conserva en ACTION, por lo que M3 no depende de PLL viva al llegar ↓clk. No se agregan temporizadores eléctricos.